Final Report

ECE 51216

Richard Rhee

Paul Larson

May 7th, 2026

1 Code Description

1.1 Davis-Putnam-Logemann-Loveland (DPLL) Algorithm

```
function DPLL(formula):
    while there is a unit clause {l},
        perform unit propagation to literal {l}
    while there is literal {l} only in positive (negative) form,
        assign {l} to 1 (0)
    if formula is empty,
        return true
    if formula contains an empty clause,
        return false
    return DPLL(formula and {l}) or DPLL(formula and {!l})
```

1.2 Heuristics Implemented

1.2.1 Maximum Occurrences in clauses of Minimum Size (MOMS)

A decision heuristic with the idea of giving preference to satisfying smaller clauses with balanced variables [1].
Each variable is scored using a formula

$$((f^*(x) + f^*(!x)) * 2^k + f^*(x) * f^*(!x) \tag{1}$$

where $f^*(x)$ is the number of unresolved smallest clauses containing x. Choose x that maximizes the score.

1.2.2 Conflict Driven Clause Learning (CDCL)

While the standard DPLL algorithm relies on chronological backtracking, we've implemented a Conflict-Driven Clause Learning (CDCL) approach to more efficiently prune the search space. When the SAT solver encounters a conflict, it perfoms conflict analysis to derive a new clause that is appended to the original set of clauses. Unlike DPLL, which was implemented recursively, our CDCL solver is iterative. This allows for non-chronological backtracking, enabling the solver to backtrack to the level where the conflicting implication started, and keep the conflict clause as small as possible by not including the literals satisfied via BCP.

Smaller conflict clauses are preferred as they are more restrictive than larger clauses and help guide the solver towards the solution faster.
The heuristic consists of two primary components:

1. **1-UIP Conflict Analysis:** When a conflict occurs, the `analyze_conflict` function identifies the root cause. It implements the **First Unique Implication Point (1-UIP)** strategy. By resolving the conflicting clause against the reason clauses in the `trail` (in reverse order of assignment), the solver derives a new learned clause. This clause represents a constraint that was previously implicit in the formula, preventing the solver from entering the same conflict state again.

2. **Backjumping:** Once a learned clause is generated, the solver calculates the **backjump level**, the second highest decision level in the learned clause. The `backjump` function then reverts the `assignment`, `assigned`, and `trail` states to that level, effectively undoing decisions that led to the conflict while retaining the newly learned knowledge.

The solver continues this loop of decision-making, BCP, and learning until either a satisfying assignment is found or a conflict at decision level 0 indicates that the formula is unsatisfiable (UNSAT).

1.3 Data Structures

After the input CNF file is parsed, we convert the data to a `numpy` array for efficient array processing.

2 Benchmarks

We created a benchmarking script that iterates through the problem sets within the repository, and also iterates on the optimizations within the SAT solver. All of the problem sets were taken from [2]. Each benchmark instance was run through our SAT solver with a timeout of 60 seconds.

2.1 uf20-91

Contains 1000 benchmark instances, all with 20 variables and 91 clauses of length 3. All problems are satisfiable. We've ommitted the table of results due to the size of the problem set.

2.1.1 DPLL

Table 1: UF20-91 - No Additional Heuristic(s) Results

Num. SAT	**Num. UNSAT**	**Num. TIMEOUT**	**Mean Runtime (s)**	**Median Runtime (s)**
1000	0	0	0.195	0.193

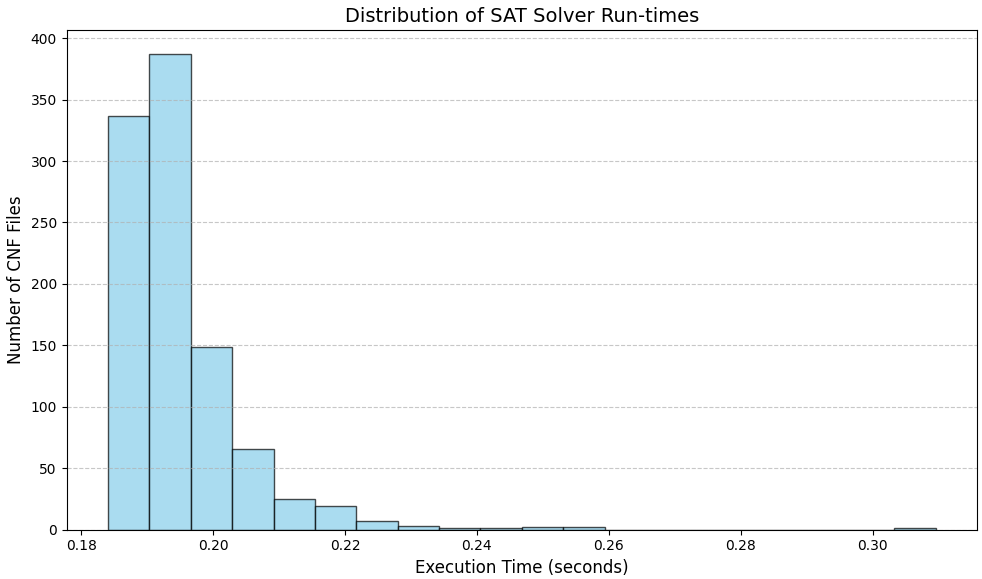

Figure 1: UF20-91 - No Additional Heuristic(s)

2.1.2 MOMS

Table 2: UF20-91 - MOMS Heuristic(s) Results

Num. SAT	**Num. UNSAT**	**Num. TIMEOUT**	**Mean Runtime (s)**	**Median Runtime (s)**
1000	0	0	0.189	0.184

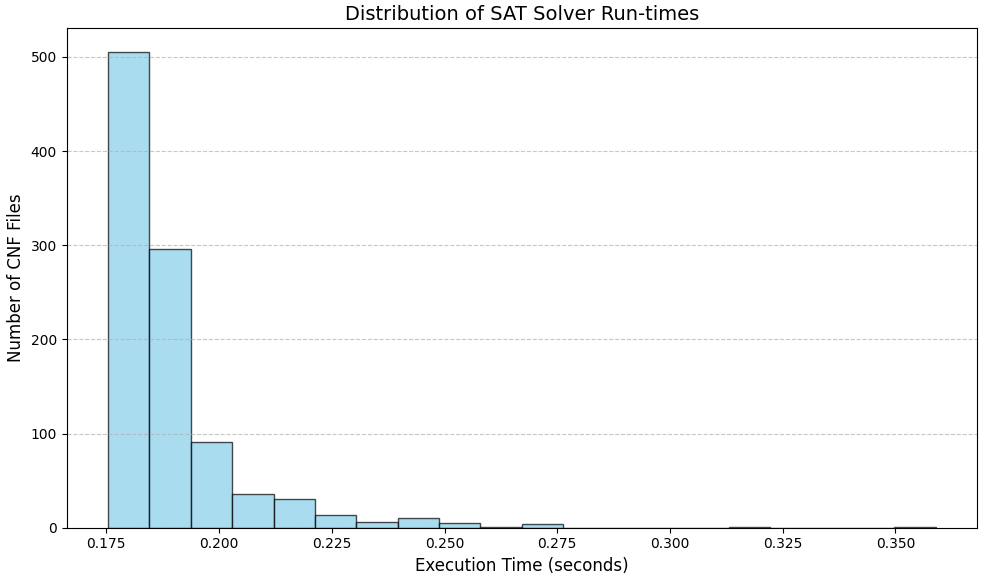

Figure 2: UF20-91 - MOMS Heuristic(s)

2.1.3 CDCL

Table 3: UF20-91 - CDCL Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
1000	0	0	0.194	0.191

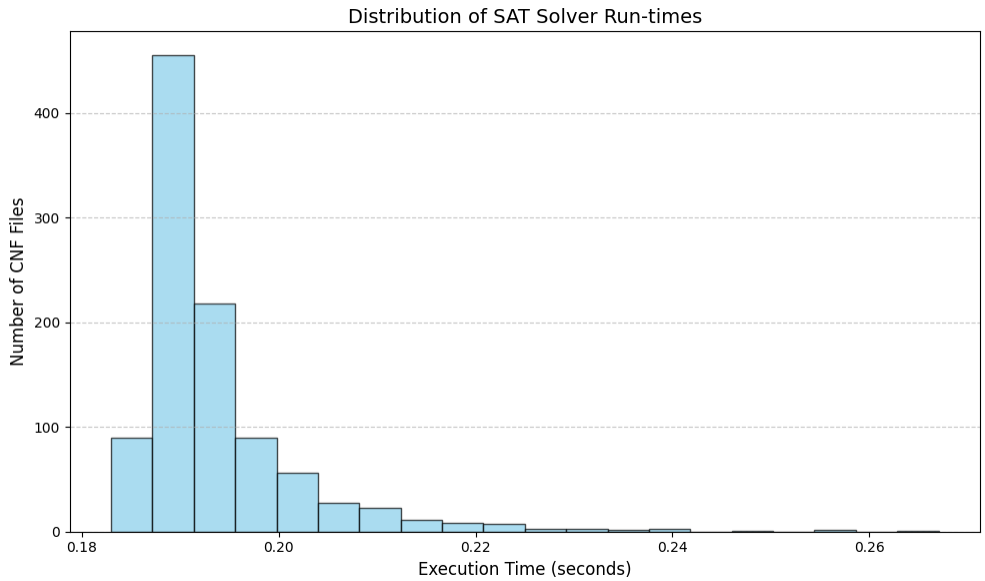

Figure 3: UF20-91 - CDCL Heuristic(s)

2.1.4 CDCL + MOMS

Table 4: UF20-91 - CDCL + MOMS Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
1000	0	0	0.191	0.190

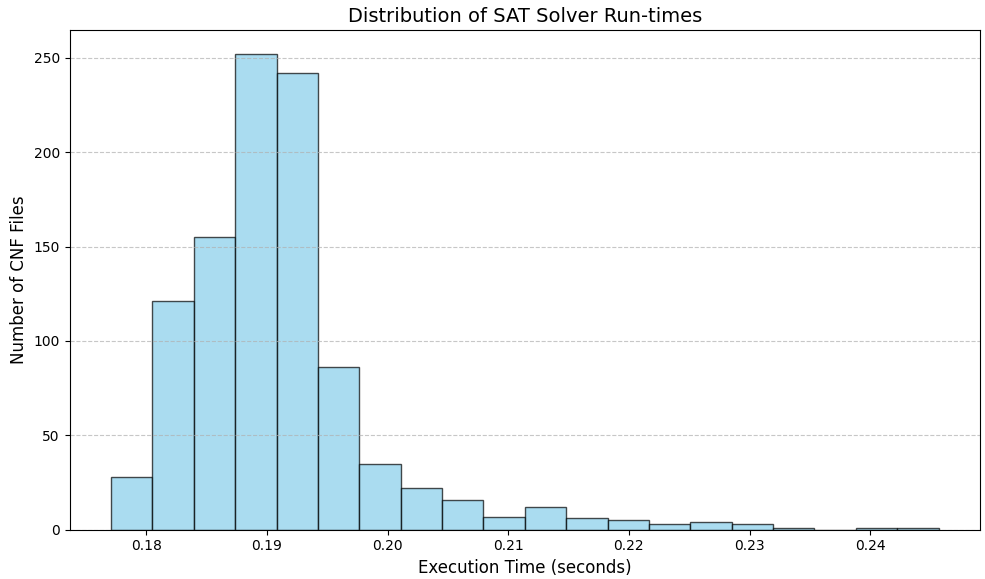

Figure 4: UF20-91 - CDCL + MOMS Heuristic(s)

2.1.5 Conclusion

The MOMS heuristic and CDCL heuristic show improvement over base DPLL, but in different ways. MOMS shows that many more cases fall into the shortest execution time bucket, going just over half of the problems. However, there are edge cases that took longer than base DPLL. CDCL shows improvement in all cases, with longest execution time being shorted to under 0.26 seconds. CDCL + MOMS shows improvement of best of both heuristics, with even more cases falling under 0.2 seconds, and the longest execution time coming under 0.25 seconds.

2.2 uf75-325

Contains 100 benchmark instances, all with 75 variables and 325 clauses of length 3. All problems are satisfiable.

2.2.1 DPLL

Table 5: UF75-325 - No Additional Heuristic(s) Results

Num. SAT	**Num. UNSAT**	**Num. TIMEOUT**	**Mean Runtime (s)**	**Median Runtime (s)**
100	0	0	0.355	0.289

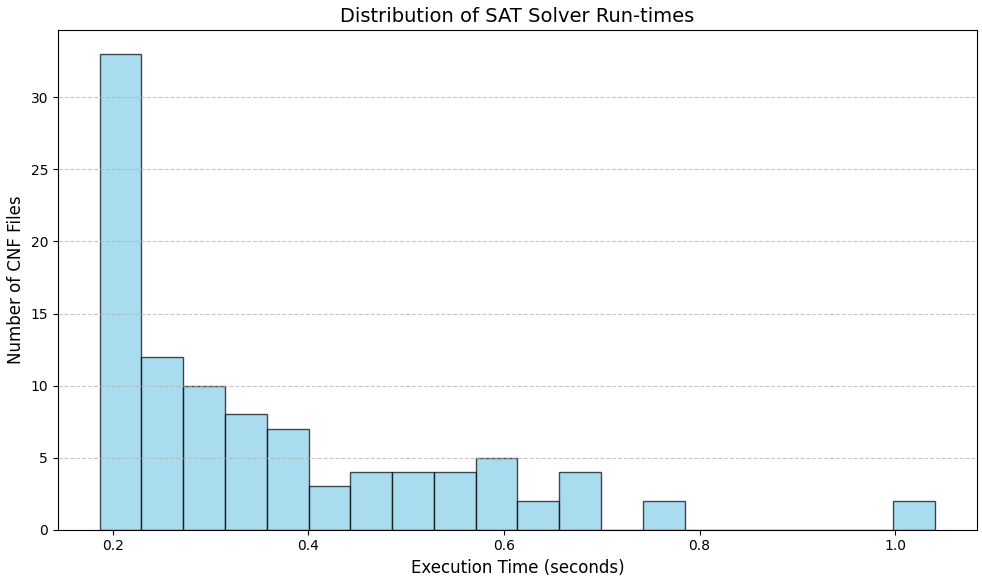

Figure 5: UF75-325 - No Additional Heuristic(s)

2.2.2 MOMS

Table 6: UF75-325 - MOMS Heuristic(s) Results

Num. SAT	**Num. UNSAT**	**Num. TIMEOUT**	**Mean Runtime (s)**	**Median Runtime (s)**
100	0	0	0.369	0.277

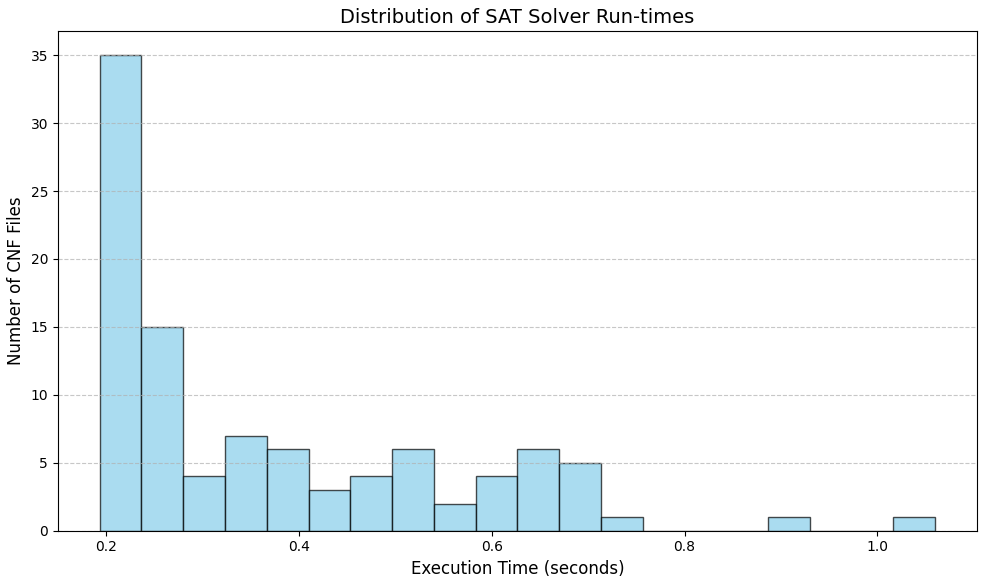

Figure 6: UF75-325 - MOMS Heuristic(s)

2.2.3 CDCL

Table 7: UF75-325 - CDCL + MOMS Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
100	0	0	0.387	0.298

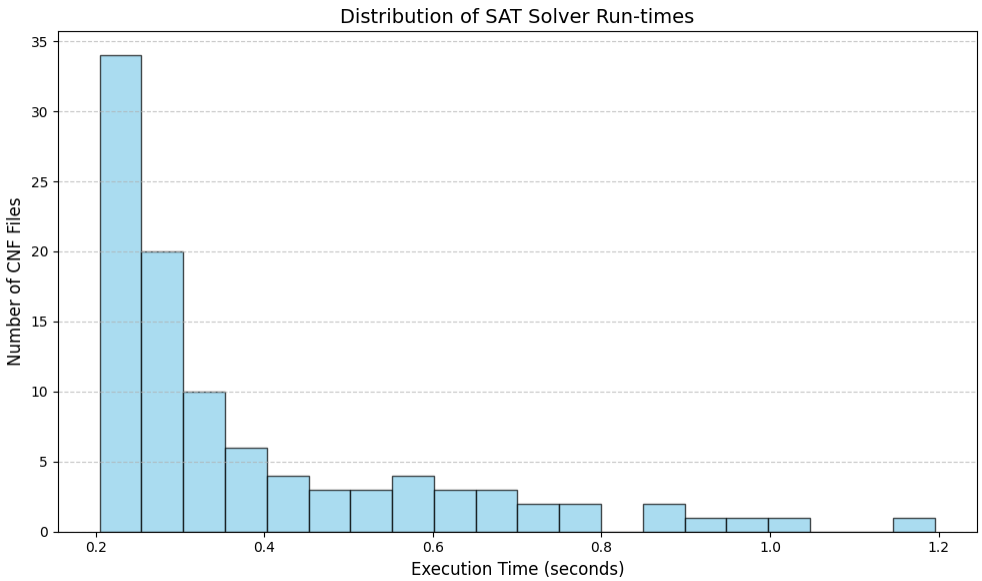

Figure 7: UF75-325 - CDCL Heuristic(s)

2.2.4 CDCL + MOMS

Table 8: UF75-325 - CDCL + MOMS Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
100	0	0	0.384	0.315

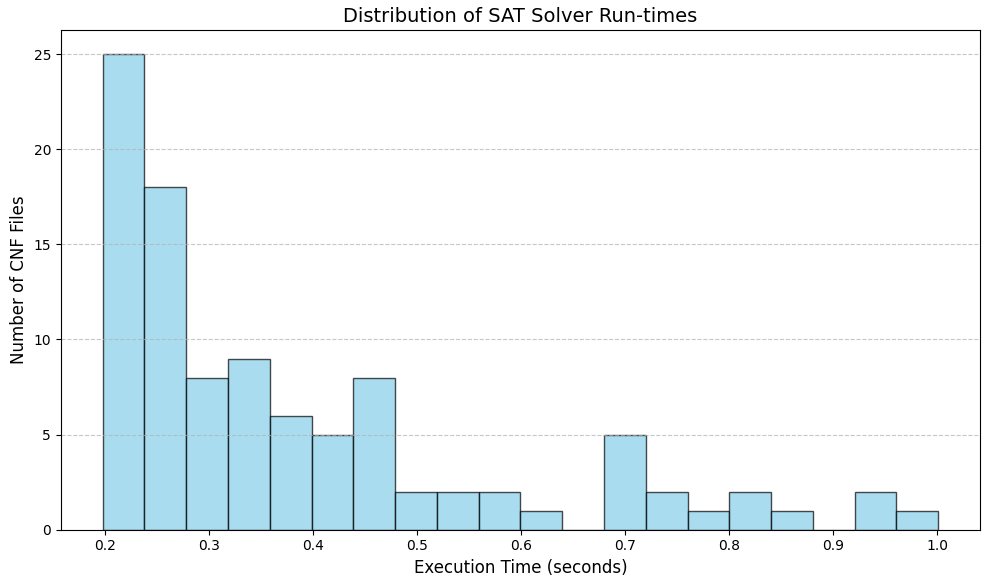

Figure 8: UF75-325 - CDCL + MOMS Heuristic(s)

2.2.5 Conclusion

MOMS shows a slight improvement over the base algorithm on their own, with more problems skewing towards the lowest end. CDCL performed slightly worse than the base DPLL. Interestingly, CDCL + MOMS shows fewer problems in the leftmost bucket, but with overall more cases taking under 0.4 seconds. CDCL + MOMS also shows better edge case handling with all cases falling under 1 second, where base DPLL, DPLL + MOMS, and DPLL + CDCL all had edge cases go over 1 second.

2.3 uuf75-325

Contains 100 benchmark instances, all with 75 variables and 325 clauses of length 3. All problems are unsatisfiable.

2.3.1 DPLL

Table 9: UUF75-325 - No Additional Heuristic(s) Results

Num. SAT	**Num. UNSAT**	**Num. TIMEOUT**	**Mean Runtime (s)**	**Median Runtime (s)**
0	100	0	0.719	0.676

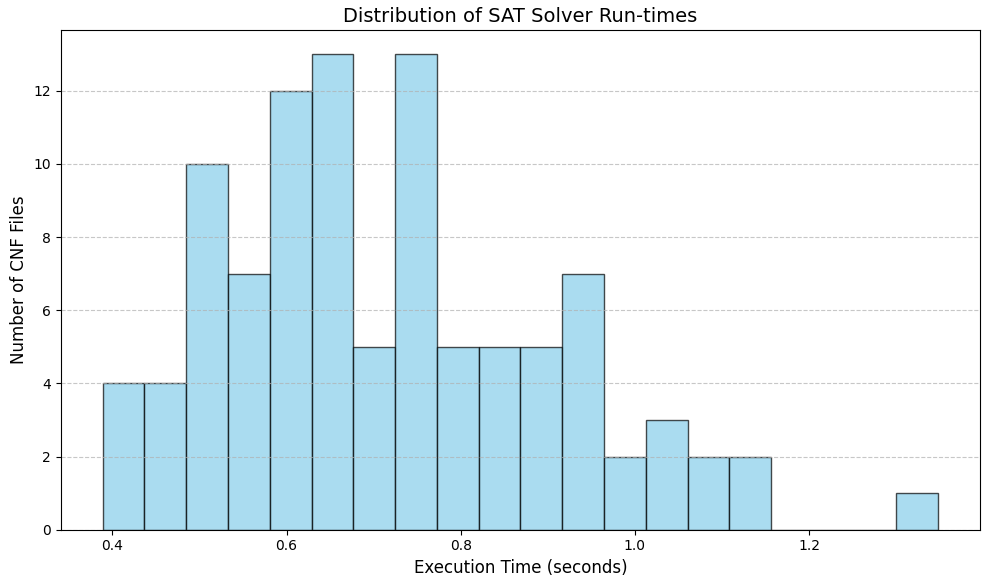

Figure 9: UUF75-325 - No Additional Heuristic(s)

2.3.2 MOMS

Table 10: UUF75-325 - MOMS Heuristic(s) Results

Num. SAT	**Num. UNSAT**	**Num. TIMEOUT**	**Mean Runtime (s)**	**Median Runtime (s)**
0	100	0	0.725	0.703

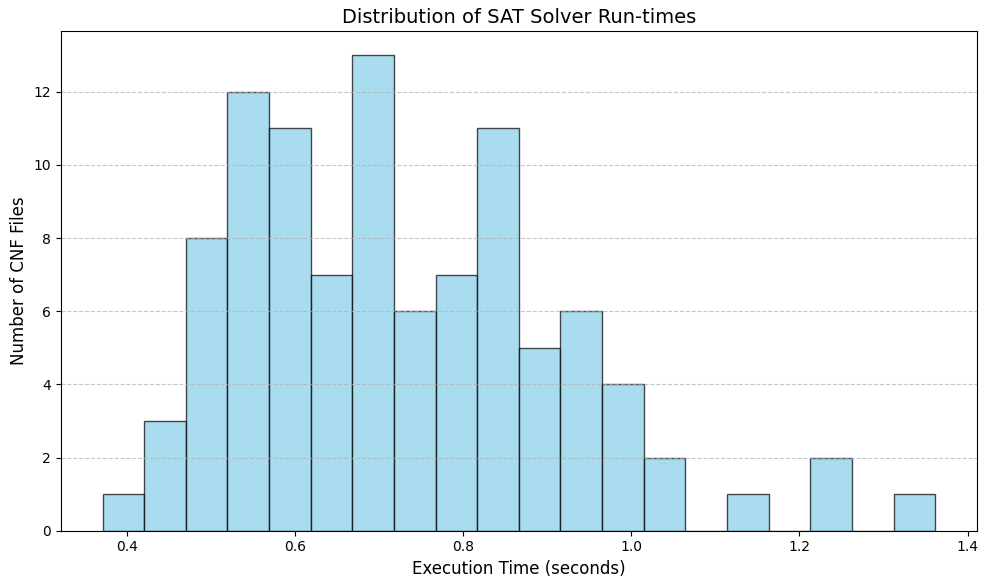

Figure 10: UUF75-325 - MOMS Heuristic(s)

2.3.3 CDCL

Table 11: UUF75-325 - CDCL Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
0	100	0	0.692	0.647

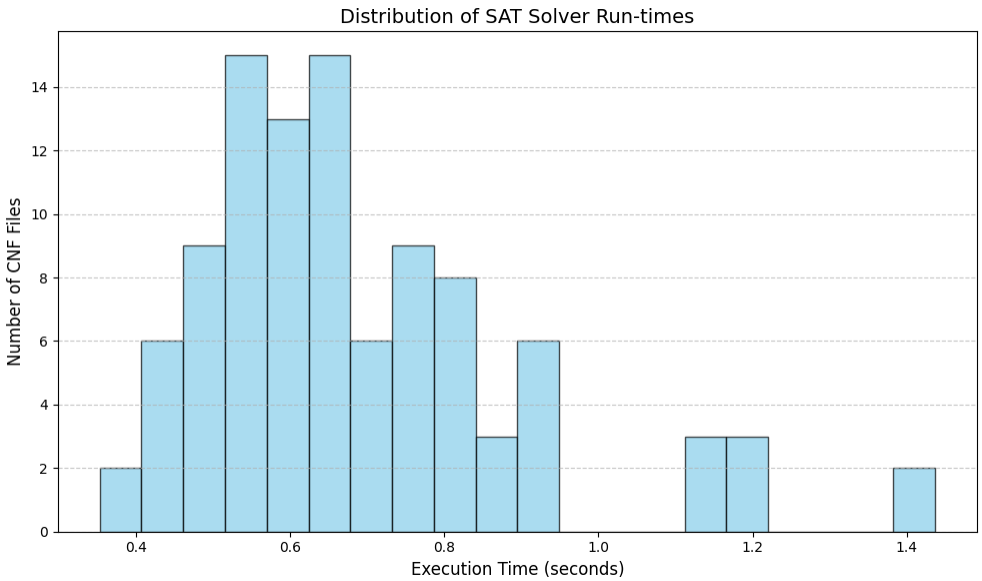

Figure 11: UUF75-325 - CDCL Heuristic(s)

2.3.4 CDCL + MOMS

Table 12: UUF75-325 - CDCL + MOMS Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
0	100	0	0.691	0.664

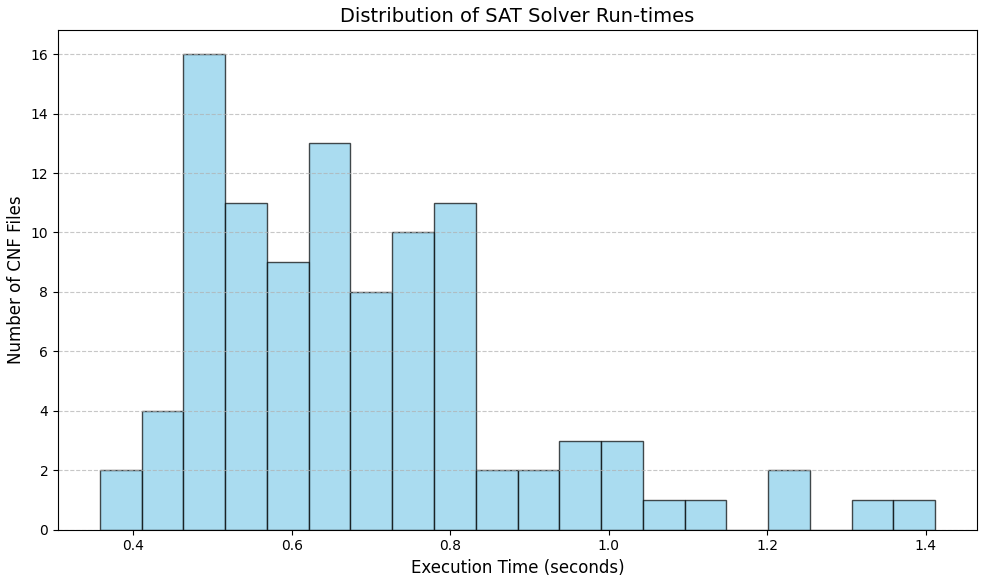

Figure 12: UUF75-325 - CDCL + MOMS Heuristic(s)

2.3.5 Conclusion

For unsatisfiable problems with equivalent length clauses, the overhead of score calculation for MOMS hurts the execution time for both DPLL and CDCL. This is as expected, as MOMS is more effective with problems of varying length clauses, and the decision it does provide for same length clauses does not outperform the cost of score calculation. CDCL performs slightly better over base DPLL, with most cases coming under 1 second. However, there are edge cases where CDCL performs worse.

2.4 JNH

Contains 50 benchmark instances. 40 have 100 variables and 800 clauses. The other 10 have 100 variables and 900 clauses. Clauses are of varying lengths. The benchmark instances are randomly generated and the set is called *Random P-SAT*.

2.4.1 DPLL

Table 13: JNH - No Additional Heuristic(s) Results

Num. SAT	**Num. UNSAT**	**Num. TIMEOUT**	**Mean Runtime (s)**	**Median Runtime (s)**
16	34	0	1.213	0.747

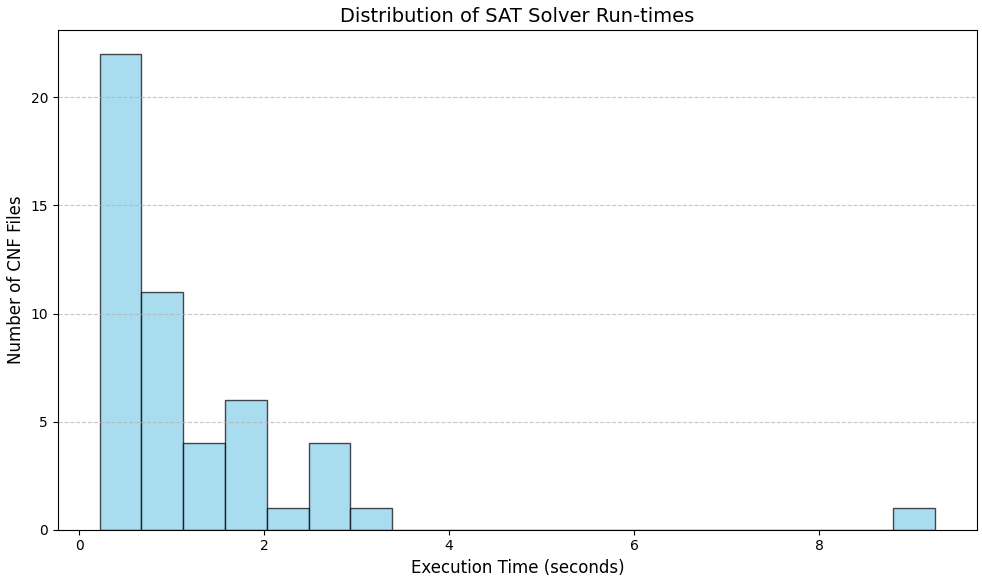

Figure 13: JNH - No Additional Heuristic(s)

2.4.2 MOMS

Table 14: JNH - MOMS Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
16	34	0	1.131	0.703

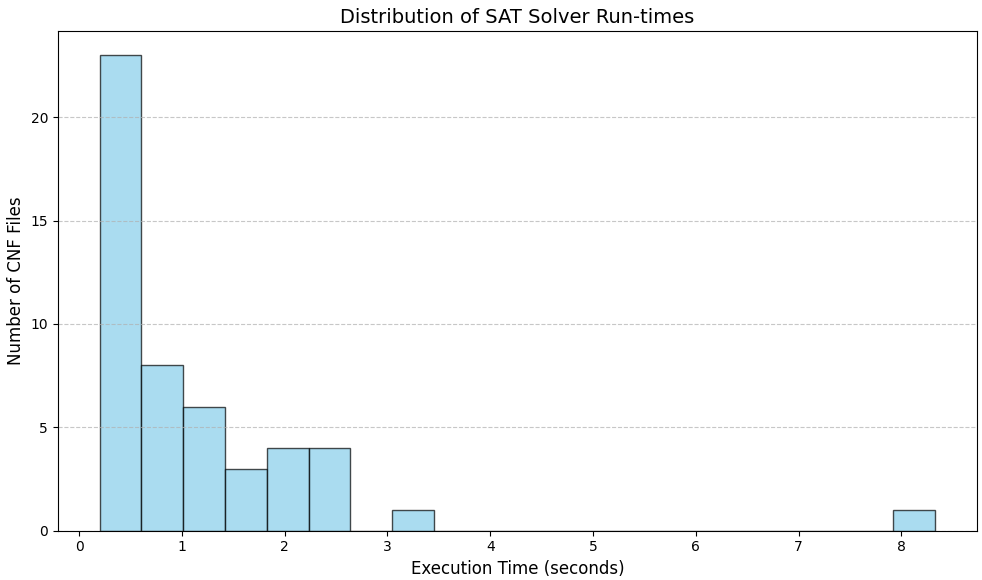

Figure 14: JNH - MOMS Heuristic(s)

2.4.3 CDCL

Table 15: JNH - CDCL Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
16	34	0	0.952	0.631

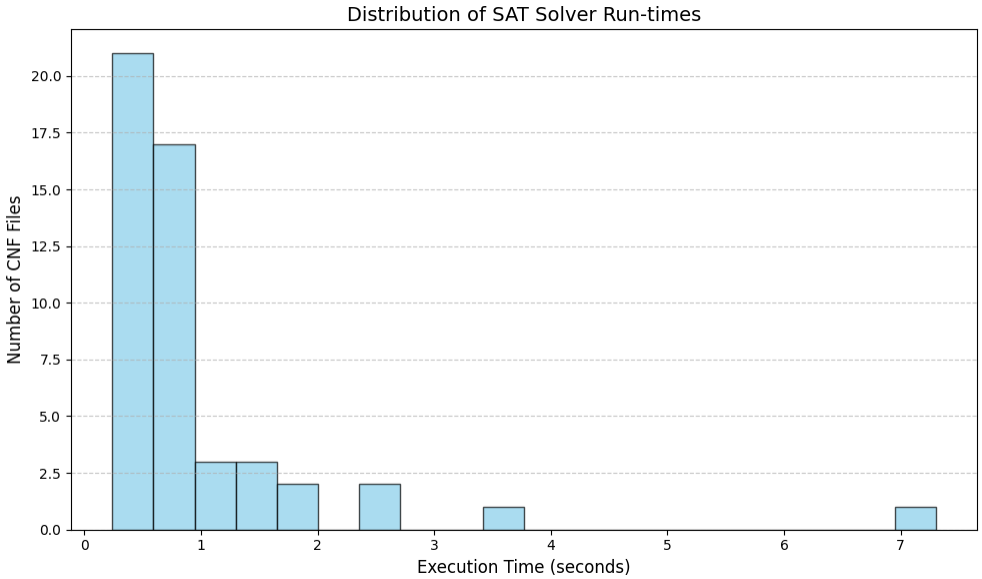

Figure 15: JNH - CDCL Heuristic(s)

2.4.4 CDCL + MOMS

Table 16: JNH - CDCL + MOMS Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
16	34	0	0.739	0.527

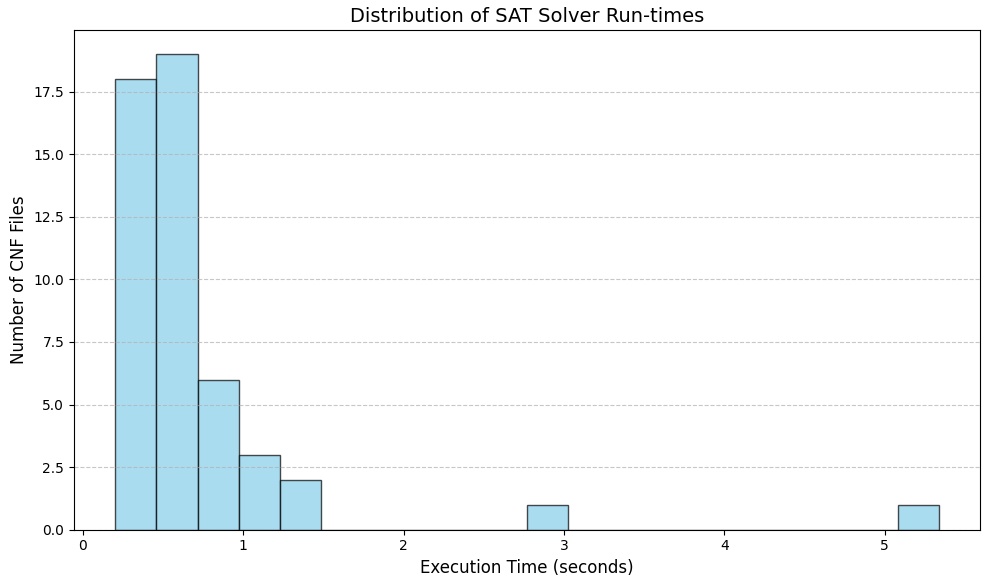

Figure 16: JNH - CDCL + MOMS Heuristic(s)

2.4.5 Conclusion

Base DPLL algorithm shows most problems taking less than 4 seconds, with a few cases going over 8+ seconds. MOMS reduces execution time by about a second with many cases, bringing the edge cases to around 8 seconds. CDCL shows a bigger improvement, with the majority of problems now falling under 1 second execution time and also bringing edge cases to around 7 seconds. Using MOMS with CDCL shows further improvement, with the vast majority of problems falling under 1 second, a few just under 3 seconds, and the edge cases now taking just over 5 seconds.

2.5 DUBOIS

Contains 13 benchmarks instances with clauses of length 3 that are all randomly generated and unsatisfiable.

2.5.1 DPLL

Table 17: DUBOIS - No Additional Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
0	0	13	60.000	60.000

All benchmark instances with the base DPLL algorithm timed-out, thus no plot was generated.

2.5.2 MOMS

Table 18: DUBOIS - MOMS Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
0	0	13	60.000	60.000

All benchmark instances with the base DPLL algorithm timed-out, thus no plot was generated.

2.5.3 CDCL

Table 19: DUBOIS - CDCL Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
0	13	0	3.118	1.797

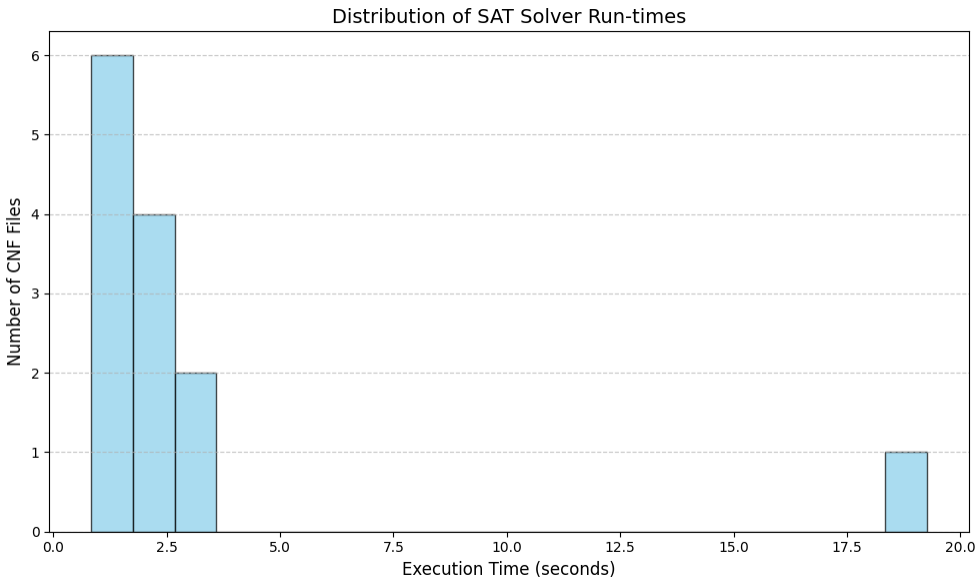

Figure 17: DUBOIS - CDCL Heuristic(s)

2.5.4 CDCL + MOMS

Table 20: DUBOIS - CDCL + MOMS Heuristic(s) Results

Num. SAT	Num. UNSAT	Num. TIMEOUT	Mean Runtime (s)	Median Runtime (s)
0	13	0	2.897	1.796

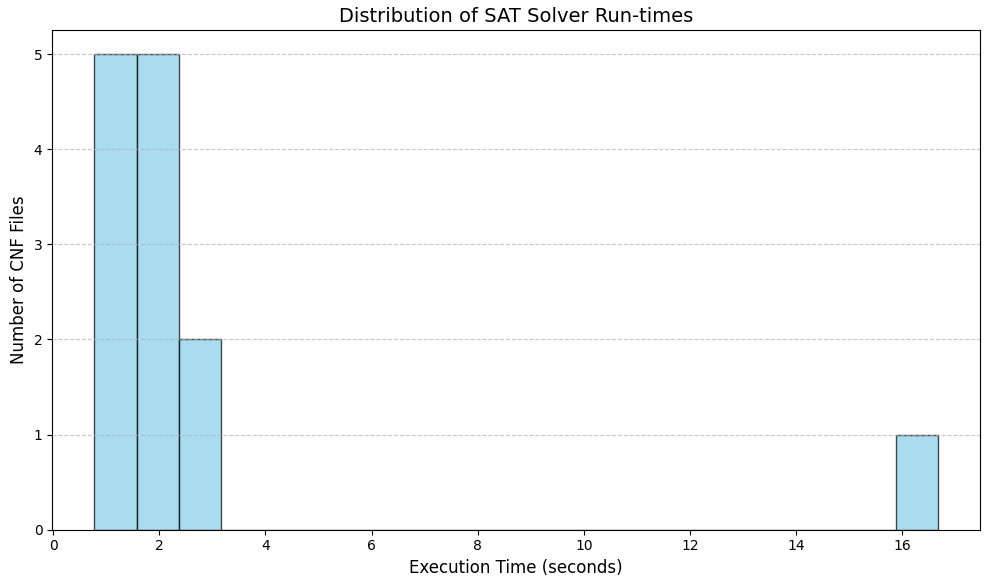

Figure 18: DUBOIS - CDCL + MOMS Heuristic(s)

2.5.5 Conclusion

All benchmark instances with the base DPLL algorithm and DPLL + MOMS both time out. CDCL does most of the heavy lifting by pruning the search space allowing the solver to complete within the 60 second timeout limit. MOMS does help with execution when used with CDCL, but not significantly.

3 General Conclusion

The MOMS heuristic shows consistent incremental improvement in all cases. There is some overhead in score calculation that shows in edge cases of unsatisfiable problems, but in most cases is negligible enough that it is worth using in all cases.

The CDCL heuristic shows the most impact in the harder problem sets, especially DUBOIS. With the harder problems, the algorithm encounters many conflicts and with CDCL, we are able to extract value from encountering a conflict. Each added conflict clause constrains which set of values leads to valid solution. In addition, the non-chronological backtracking enables the algorithm to exit a part of the search space that does not contain a valid solution faster.

With CDCL, there is a downside when the problem set is trivial, as seen with the uf20-91 and uf75-325. The cost of the additional overhead outweighs the value added by the heuristic.

When combined, we see the best of both heuristics. MOMS provides an intelligent branching strategy that guides the solver toward high-impact variables, while CDCL ensures the solver learns from conflicts, pruning the search space and preventing the algorithm from stagnating in a part of the search space that won't yield a valid solution. This synergy results in a solver that is both proactive in its search and efficient in its recovery.

References

[1] E. M. Clarke and D. Kroening. "Heuristics for efficient sat solving. "[Online]. Available: https://www.cs.cmu.edu/~emc/15-820A/reading/sat_cmu.pdf. (accessed: 05.06.2026).

[2] H. H. Hoos and T. Stützle, "SATLIB: An online resource for research on SAT," in *SAT 2000*, I. P. Gent, H. van Maaren, and T. Walsh, Eds., IOS Press, 2000, pp. 283–292. [Online]. Available: http://www.satlib.org.